RESTAURANT APP MVP

Software Requirements Specification · Version 2.0

React Native · Node.js / Express · MongoDB

Assignment 2 — Fall 2026

Scope: Existing A1 mobile app upgraded with a shared REST API and persistent database. Real payments and push notifications remain out of scope.

1. Introduction

1.1 Purpose

This SRS v2 updates the A1 requirements for the Restaurant App MVP. It defines the mobile client, Express API, MongoDB data model, authentication, order and reservation rules, API endpoints, and integration expectations. It is intended for the student developer, evaluator, testers, and portfolio reviewers.

1.2 Scope

In scope: React Native/Expo screens retained from A1; Express server; MongoDB persistence through Mongoose; customer registration/login; bcrypt password hashing; one-day JWT authentication; public menu search/category filtering; manager-only menu CRUD; table listing; order creation and history; manager order status changes; reservation creation, history and manager decisions; input validation; error handling; Postman testing; and cross-device synchronization when clients reach the same server.

Out of scope: real payment processing, push notifications, production-grade delivery dispatch, third-party POS integrations, and live deployment to a public production environment.

A1 limitations and A2 resolution: (1) Manager menu changes did not reach other phones → read/write shared MongoDB-backed menu endpoints. (2) Orders/bookings were device-local → persist records on the server. (3) Reinstalling could remove local records → store business records in MongoDB. (4) Client totals could be manipulated → calculate totals from database prices on the server. (5) Reservation availability could differ by device → validate table capacity and existing bookings on the server. (6) UI-only manager restrictions could be bypassed → enforce JWT role checks in middleware.

1.3 Definitions and acronyms

Term	Definition
SRS	Software Requirements Specification.
MVP	Minimum Viable Product: the smallest useful working version.
API	Application Programming Interface used by the app to communicate with the server.
REST	HTTP-based resource-oriented API style.
Endpoint	A method and URL that performs an API operation.
MongoDB / Collection	Document database / group of related documents.
Mongoose	ODM used to define schemas, validate data, and query MongoDB.
JWT	Signed token used to authenticate protected API requests.
Middleware	Function that runs during the Express request/response pipeline.
JSON	Data interchange format used in request and response bodies.
HTTP status code	Number indicating request outcome, such as 200, 201, 400, 401, 403, 404, or 500.
CORS	Browser/client origin policy configured by the API.

2. Overall Description

2.1 User roles

Role	Permissions and goals
Customer	Register/login; browse and search menu; place an order; view own orders; create and view own reservations; cancel own reservation where allowed.
Manager	All customer-visible menu browsing; create/update/delete menu items; view all orders and reservations; advance valid order statuses; accept/decline reservations; manage tables.
System	Validate payloads, verify JWTs, enforce access, calculate trusted prices, persist records, and return consistent errors.

2.2 User stories

• As a customer, I want to create an account so that my orders are associated with me.

• As a customer, I want to search and filter menu items so that I can find food quickly.

• As a customer, I want the server to use current menu prices so that my total is accurate.

• As a customer, I want to view my orders from another device so that my history is consistent.

• As a customer, I want to reserve a table so that availability is checked centrally.

• As a customer, I want to see manager-updated order status without reinstalling the app.

• As a manager, I want to mark a dish unavailable so every customer sees the change.

• As a manager, I want to view all orders so I can manage restaurant operations.

• As a manager, I want to accept or decline reservations so the booking process is controlled.

• As a manager, I want protected endpoints so customers cannot edit menu or order statuses.

3. Functional Requirements

ID	Requirement
FR-01	The system shall expose GET /api/health with status and current time.
FR-02	The system shall register customer accounts and reject duplicate email addresses.
FR-03	The system shall hash passwords with bcrypt and never return password hashes.
FR-04	The system shall issue a JWT that expires after one day on successful login.
FR-05	The system shall allow public menu listing, category filtering, and case-insensitive name search.
FR-06	The system shall allow manager-only create, update, and delete menu operations.
FR-07	The system shall validate MongoDB IDs and return 400 for malformed IDs.
FR-08	The system shall list active tables and their seating capacities.
FR-09	The system shall require authentication to place an order.
FR-10	The system shall read current item prices from MongoDB when calculating order totals.
FR-11	The system shall reject orders containing missing or unavailable menu items.
FR-12	The system shall let customers retrieve only their own orders.
FR-13	The system shall let managers retrieve all orders.
FR-14	The system shall enforce valid order status transitions on the server.
FR-15	The system shall validate reservation date, time slot, party size, phone, and table capacity.
FR-16	The system shall reject a reservation when the selected table/time is already booked.
FR-17	The system shall let customers retrieve their own reservations.
FR-18	The system shall let managers view and accept/decline reservations.
FR-19	The system shall return JSON errors and a 404 response for unknown routes.
FR-20	The system shall persist menu, user, table, order, and reservation records in MongoDB.

4. Non-Functional Requirements

Category	Requirement
Security	Passwords use bcrypt; protected routes require a verified JWT; manager operations require manager role; secrets stay in .env.
Performance	Simple API reads should generally respond within 1 second on a local development network under normal test load.
Reliability	Database connection failures must stop startup with a clear error; API errors must return JSON.
Maintainability	Routes, controllers, models, and middleware are separated; reusable client request logic is in src/api/client.js.
Data integrity	Mongoose schema constraints and server-side business rules validate data before saving.
Compatibility	Expo app can connect from an emulator or phone using the correct laptop IP and network/firewall settings.
Usability	Screens preserve loading/error feedback; server validation messages should be shown in a readable form.

5. Database Design

Collection	Key fields and rules	Relationships
Users	name:String required; email:String required, unique, lowercase; password:String required, bcrypt hash, not selected by default; role:customer|manager; timestamps.	Referenced by Order.user and Reservation.user.
MenuItems	name:String required; description:String; price:Number required min 0; category enum; image:String; isSpecial:Boolean; isAvailable:Boolean.	Referenced by each Order item through menuItem; order snapshots name and unit price.
Tables	label:String required unique; seats:Number min 1; isActive:Boolean.	Referenced by Reservation.table and optional Order.table.
Reservations	user:ObjectId required; table:ObjectId required; date:Date; time:String; partySize 1–12; contactName; phone; status enum; timestamps.	References Users and Tables.
Orders	user:ObjectId; items array with menuItem, name, unitPrice, quantity; subtotal, serviceCharge, tax, discount, total; type; status enum; timestamps.	References Users, MenuItems, and optionally Tables.

6. UML Diagrams

The A1 use case, class, component and state diagrams remain in A1/UML. A2 adds an updated order sequence and deployment diagram in A2/UML. The sequence shows the mobile app calling Express, the server validating the JWT and current menu prices, MongoDB saving the order, and the API returning the saved order. The deployment diagram shows the phone, laptop-hosted API, and MongoDB Atlas or local database. PlantUML source files are included for editing and export.

7. Updated Frontend and Server Integration

The React Native app uses a shared API request helper in src/api/client.js. The BASE_URL must be changed for the actual development network. Authentication responses store the token and user in AsyncStorage; requests include Authorization: Bearer token when available. The menu context loads from GET /api/menu. A reusable useApi hook exposes data, loading, error, and refetch. The remaining screen integrations must be verified against the live API before final demonstration.

8. API Endpoints

Method	Endpoint	Access	Purpose
GET	/api/health	Public	Health check
POST	/api/auth/register	Public	Register customer
POST	/api/auth/login	Public	Login / JWT
GET	/api/menu	Public	List/search/filter
GET	/api/menu/:id	Public	Get menu item
POST	/api/menu	Manager	Create item
PUT	/api/menu/:id	Manager	Update item
DELETE	/api/menu/:id	Manager	Delete item
GET	/api/tables	Public	List active tables
POST	/api/tables	Manager	Create table
POST	/api/orders	Authenticated	Place order
GET	/api/orders/my	Authenticated	Own orders
GET	/api/orders	Manager	All orders
PATCH	/api/orders/:id/status	Manager	Update status
POST	/api/reservations	Authenticated	Create booking
GET	/api/reservations/my	Authenticated	Own bookings
GET	/api/reservations	Manager	All bookings
PATCH	/api/reservations/:id	Authenticated/Manager	Cancel or decide booking

9. Verification and Acceptance

■ Health endpoint returns 200 and status ok.

■ Valid login returns a token and user without password data.

■ No-token menu mutation returns 401; customer-token mutation returns 403; manager-token mutation succeeds with valid input.

■ Menu filters/search work and malformed MongoDB IDs return 400.

■ Order totals are calculated from MongoDB prices and unavailable items are rejected.

■ Order status cannot move backwards or leave terminal states.

■ Reservation capacity and duplicate table/time booking checks are enforced.

■ Customer and manager can see shared server-side changes from separate devices.

■ Capture genuine Postman, database, and device screenshots and record the end-to-end flow.

10. Setup and Submission Notes

Backend setup: enter server/, run npm install, copy .env.example to .env, configure MONGO_URI and JWT_SECRET, run npm run seed, then npm run dev. The Expo app runs from the repository root with npm install and npx expo start. Update BASE_URL in src/api/client.js. Never commit .env. The included Postman collection is A2/restaurant-api.postman_collection.json. Real screenshots and the required screen recording must be added after the app is run and tested.